

Semester Project

Bayan: A Serverless Study Guide



CE-308

Instructor: Ms. Safia Baloch

2023330	Mohammad Musa Ali
2023753	Wali Muhammad Khan
2023176	Dara Shikoh Bodla

1. Executive Summary

Bayan is a serverless, event-driven AI platform that turns recorded lectures into searchable, bilingual study guides without a human in the loop. A student uploads an audio file; the system transcribes it, extracts key topics, translates the content into Urdu, synthesizes an Urdu audio summary, and pushes live progress updates to the browser over a WebSocket connection. The entire pipeline runs on AWS managed services and costs \$0.00 under the Free Tier for the scale of this project.

The project is intentionally a departure from a conventional three-tier web application. There is no EC2 instance, no load balancer, no always-on server. Every piece of compute executes only in response to an event — a file landing in S3, a state machine advancing, a record appearing in a DynamoDB stream. This is the architectural pattern behind modern production systems at scale, and building it is the core learning goal of CE 313.

The team has budgeted three weeks and split the work into three non-overlapping tracks so that all three members can work in parallel from day one. A previous coursework project (UniEvent, built on EC2 + CloudFormation) established the team's baseline AWS fluency; Bayan is designed to exceed that baseline on every axis: more services, more automation, and a change in basic assumptions from request-driven to event-driven computing.

2. Problem Statement & Motivation

Students at GIKI routinely record lectures but rarely revisit them. A two-hour recording is unsearchable, unmemorizable, and in the case of content that mixes English with Urdu explanations, often inaccessible to students who are stronger in one language than the other. The problem is not the absence of AI tools — Transcribe, Comprehend, Translate and Polly exist. The problem is that no student-facing workflow stitches them together with proper orchestration, authentication, and delivery.

Bayan addresses this gap. More importantly, building Bayan forces the team to engage with the architectural questions that commercial ML pipelines face: how do you hand off a 50 MB file to the cloud without timing out an API Gateway? How do you retry a failed transcription job without re-running the whole pipeline? How do you tell a user their lecture is 40% processed without asking them to refresh the page? These are the questions this project is designed to answer.

3. Objectives

The project has four primary technical objectives and one demonstration objective.

- **Event-driven orchestration:** Build a complete pipeline in which no component polls or blocks. Every transition is triggered by an event (S3 Put, DynamoDB Stream record, Step Functions state transition).
- **Managed-ML integration:** Integrate four AWS ML services (Transcribe, Comprehend, Translate, Polly) through Step Functions so that failure in any single stage is retried automatically with exponential backoff.
- **Real-time client feedback:** Push per-stage progress updates to an authenticated web client via API Gateway's WebSocket mode, demonstrating full-duplex communication over AWS managed infrastructure.
- **Zero-cost delivery:** Keep the production deployment entirely inside the AWS Free Tier by avoiding any always-on computer (no EC2, no NAT Gateway, no RDS).
- **Demonstrable artefact:** Deliver a live demo in which a 3–5-minute audio file is uploaded, and a complete bilingual study guide is returned in under two minutes, with the Step Functions visual execution graph displayed alongside the frontend.

4. Scope

4.1 In Scope

- Audio upload via Cognito-authenticated web client, using S3 pre-signed URLs (files up to 100 MB).
- Automatic transcription (up to 4 speakers) using Amazon Transcribe.
- Key phrase, entity, and sentiment extraction using Amazon Comprehend.
- English-to-Urdu translation using Amazon Translate.
- Urdu/Hindi audio synthesis of the summary using Amazon Polly with SSML tuning.
- Full pipeline orchestration via AWS Step Functions, including retry and catch states.
- Live progress updates pushed the client via API Gateway WebSocket and DynamoDB Streams.
- Persistent history of processed lectures with key phrase-based search, stored in DynamoDB with a Global Secondary Index.
- Email notification on pipeline completion or failure via Amazon SNS.
- Static frontend hosted on S3 and distributed via CloudFront, with the full stack deployed via AWS SAM.

4.2 Out of Scope

- Mobile native clients (iOS/Android). The front end is web-only.
- Third-party integrations (Google Drive, Dropbox, LMS hooks).
- Multi-tenant billing or subscription management.

- Generative-LLM-based summarization (Comprehend key phrases are used instead to keep the stack fully AWS-native and free-tier).

5. System Architecture

The system follows a strict event-driven topology. The diagram below describes the data flow from the moment a student clicks 'Upload' to the moment they receive the completion email.

Logical Data Flow

#	Stage	What Happens
1	Upload	Client requests a pre-signed URL from API Gateway → Lambda → uploads directly to the raw-audio S3 bucket.
2	Trigger	S3 PutObject event triggers a Starter Lambda that creates a DynamoDB record and invokes a Step Functions execution.
3	Transcribe	Step Functions calls Amazon Transcribe. Execution waits (via task taken) until the job completes.
4	Analyze (Parallel)	Two branches run in parallel: (a) Comprehend DetectKeyPhrases + DetectEntities + DetectSentiment, (b) Translate TranslateText to Urdu.
5	Synthesize	Polly StartSpeechSynthesisTask generates an Urdu/Hindi audio summary and writes the MP3 to the processed-notes S3 bucket.
6	Persist	A Finalizer Lambda writes transcript, summary, key phrases, and audio URL to DynamoDB with status = 'completed'.
7	Notify	DynamoDB Streams emitted the update. A Broadcaster Lambda pushes the progress event to the user's WebSocket connection and SNS sends the completion email.

5.1 AWS Services Used

Service	Purpose in Bayan
Amazon S3	Two buckets: raw-audio (uploads) and processed-notes (Polly output, static frontend assets).
AWS Lambda	Eight functions: pre-signed URL generator, starter, transcribe-launcher, transcribe-callback, comprehend-analyzer, translate-worker, Polly-launcher, finalizer, WebSocket connect/disconnect/broadcast.
AWS Step Functions	Central state machine with parallel branches, retry policies, and catch handlers for pipeline failure.
Amazon Transcribe	Speech-to-text (ShowSpeakerLabels = true, MaxSpeakerLabels = 4).
Amazon Comprehend	Key phrase, entity, and sentiment extraction on the transcript.

Service	Purpose in Bayan
Amazon Translate	English → Urdu translation of the summary.
Amazon Polly	Audio synthesis of the Urdu summary using the Kajal (Hindi) voice with SSML prosody adjustments.
Amazon DynamoDB	Single-table design with GSIs for user-history and key phrase-search queries. Streams enabled for real-time broadcast.
API Gateway (REST + WebSocket)	REST endpoint for pre-signed URL generation; WebSocket endpoint for live progress push.
Amazon Cognito	User Pool for signup/login; JWT authorizer on API Gateway.
Amazon CloudFront	CDN in front of the S3-hosted React frontend.
Amazon SNS	Email notification on pipeline completion or failure.
Amazon CloudWatch	Dashboard with Lambda error rate, Step Functions failures, Transcribe job latency; alarms on critical metrics.
AWS SAM	Infrastructure-as-code for the entire stack. One template, one deploy command.

6. Work Breakdown Structure

The three roles below are deliberately non-overlapping so that all members can progress in parallel without merge conflicts. Each owner is the final authority on their domain, but pair-programming and cross-review are expected at integration points (marked *).

6.1 Member 1 — Dara Shikoh Bodla (2023176)

Role: AI Pipeline Lead & Team Coordinator

- **Owns:** All four AI-service Lambdas (transcribe, comprehend, translate, Polly), the Step Functions state machine including retry/catch logic, the finalizer Lambda that writes to DynamoDB, SNS email configuration, and the master SAM template that stitches everything together.
- **Deliverables:** state-machine.asl.json, four Python Lambda handlers with unit tests, Step Functions execution screenshots for the demo, SNS email templates.
- **Integration points*:** DynamoDB schema (with Musa), WebSocket payload format (with Wali).

6.2 Member 2 — Muhammad Musa (2023330)

Role: Infrastructure, Security & Observability Lead

- **Owns:** Both S3 buckets with event notifications and lifecycle rules, the Cognito User Pool, and App Client, all IAM roles and policies (least-privilege, audited via IAM Access Analyzer), the pre-signed URL Lambda, the DynamoDB table definition and Stream configuration, and the CloudWatch dashboard with alarms.

- **Deliverables:** Complete SAM infrastructure layer, IAM policy documents, Cognito hosted UI screenshots, CloudWatch dashboard snapshot, signed-URL Lambda with expiry configuration.
- **Integration points*:** DynamoDB schema (with Dara), Cognito JWT verification on both REST and WebSocket APIs (with Wali).

6.3 Member 3 — Wali Muhammad Khan (2023753)

Role: Frontend & Real-Time Layer Lead

- **Owns:** The React single-page app (Vite + TypeScript), Cognito auth flow on the client, the upload widget that consumes pre-signed URLs, the complete WebSocket API (connect, disconnect, broadcast routes), the Broadcaster Lambda that reads DynamoDB Streams and posts to active WebSocket connections, the progress-bar UI, and the history + key phrase-search view. Deploys the frontend to S3 + CloudFront.
- **Deliverables:** Deployed React app accessible at a CloudFront URL, WebSocket handlers, DynamoDB Streams Lambda, responsive UI with progress tracking and search.
- **Integration points*:** WebSocket payload format (with Dara), Cognito JWT verification (with Musa).

7. Timeline — Three-Week Sprint Plan

The schedule is built around three hard milestones, one per week. Each milestone is a demonstrable working system; we do not leave a week without something runnable end-to-end, even if only partially populated.

Week	Theme	Key Tasks	Milestone
Week 1	Foundation	AWS account setup, IAM bootstrapping, SAM project scaffold, S3 buckets, Cognito User Pool, pre-signed URL Lambda, React app skeleton with login, DynamoDB table.	Logged-in users can upload a file to S3 through the browser.
Week 2	AI Pipeline	Transcribe, Comprehend, Translate, Polly Lambdas; Step Functions state machine with retries and catch; finalizer Lambda; SNS email on completion.	Upload triggers full pipeline; users receive email with study guide.
Week 3	Real-Time & Polish	WebSocket API, DynamoDB Streams broadcaster Lambda, progress UI, history + key phrase search, CloudWatch dashboard, end-to-end testing, demo preparation.	Live progress updates in browser; demo-ready deployment.

8. Budget & Cost Analysis

The project is designed to run at zero cost under the AWS Free Tier for the duration of development, testing, and demonstration. The table below lists every chargeable service and the expected consumption during the three-week project.

Service	Free Tier Allowance	Expected Usage	Cost
Transcribe	60 audio min/month × 12 months	~40 minutes	\$0.00
Comprehend	50,000 units/month × 12 months	~200 units	\$0.00
Translate	2,000,000 chars/month × 12 months	~60,000 chars	\$0.00
Polly	5,000,000 chars/month × 12 months	~30,000 chars	\$0.00
Lambda	1M requests + 400k GB-sec/month (forever)	~5,000 requests	\$0.00
Step Functions	4,000 state transitions/month (forever)	~500 transitions	\$0.00
DynamoDB	25 GB storage + 25 WCU/RCU (forever)	<100 MB, <5 WCU peak	\$0.00
API Gateway	1M REST + 1M WebSocket msgs/month × 12 months	~2,000 messages	\$0.00
S3 + CloudFront + Cognito + SNS	All well within Free Tier	Negligible	\$0.00
TOTAL			\$0.00

Mitigations against accidental overspend: (a) a Lambda pre-check rejects audio files longer than 15 minutes during the development phase, (b) S3 lifecycle rules delete raw uploads after 7 days, (c) an AWS Budget alarm is configured at \$5.00 total to trigger an SNS alert, (d) no EC2 instances, NAT Gateways, or RDS databases are ever provisioned.

9. Risk Assessment

Risk	Severity	Mitigation
Polly Urdu voice quality is not production-grade; output may sound unnatural.	Medium	Validated in Day 1 of Week 1 by synthesizing a sample Urdu sentence using the Kajal (Hindi) voice. If unacceptable, fall back to English audio summary + Urdu text.
WebSocket + DynamoDB Streams integration is the most technically involved component.	Medium	Allocated a full week (Week 3) with Wali as the owner. Pair-programming session scheduled for the IAM permissions on <code>execute-api:ManageConnections</code> , which is the most common failure point.
Cognito JWT verification on API Gateway WebSocket is a common permission pitfall.	Low	Use a Lambda authorizer on the <code>\$connect</code> route rather than a native Cognito authorizer; documented reference implementation followed.
Transcribe is asynchronous and can take minutes; Step Functions must wait.	Low	Use Step Functions' task token pattern (<code>waitForTaskToken</code>) with a Transcribe completion event on EventBridge, rather than in-Lambda polling.
Accidental Free Tier overspend via an oversized test file.	Low	File-size and duration pre-check in the signed-URL Lambda; AWS Budget alarm at \$5.00.
Member illness or exam conflict during the sprint.	Medium	All code is committed daily to a shared GitHub repository. Each member writes a short README for their component so another member can pick it up. Weekly sync on Fridays to surface blockers early.

10. Deliverables

- A deployed, publicly accessible demo URL (CloudFront distribution).
- A public GitHub repository containing the full SAM template, all Lambda source code, the React frontend, and a README.
- An implementation guide (`IMPLEMENTATION_GUIDE.md`) with day-by-day build instructions.
- An architecture diagram showing all AWS services and event flows.
- A final presentation deck and live demonstration.
- A CloudWatch dashboard showing real metrics from end-to-end runs.
- This proposal document.

11. Expected Outcomes

By the end of the three-week sprint, the team will have built, deployed, and demonstrated a production-shape serverless application that uses ten distinct AWS services working together through event triggers.

The project will serve as an individual portfolio artefact for each member — Dara will have a state-machine-driven AI pipeline to show for AI-focused internships, Musa will have a hardened security and observability layer for infrastructure roles, and Wali will have a real-time WebSocket application for frontend or full-stack roles.

The course-level outcome is a demonstrable shift in how the team thinks about cloud computing: away from the request-driven, always-on, server-centric model of Assignment 1 (UniEvent) and toward the event-driven, stateless, managed-service model that defines modern cloud architecture.

12. Conclusion

Bayan is ambitious but scoped. Every feature in this proposal has been cost-validated against the AWS Free Tier, every risky component has been identified and assigned a mitigation, and every member has a non-overlapping track that they can work on from day one. The project is intentionally harder than UniEvent, not in terms of hours, but in terms of the architectural concepts it forces the team to engage with: asynchronous orchestration, event-driven triggers, full-duplex client communication, and least-privilege security across a mesh of managed services. We believe these are the concepts CE 313 is fundamentally about, and we are ready to build.